

UNIVERSIDADE PRESBITERIANA MACKENZIE

LEONARDO RODRIGUES GENEROSO

**PADRÃO EVOLUTIVO DE ARQUIVOS CSS: um estudo em projetos de código
aberto utilizando Mineração de Repositório de Software**

São Paulo

2015

LEONARDO RODRIGUES GENEROSO

**PADRÃO EVOLUTIVO DE ARQUIVOS CSS: um estudo em projetos de código
aberto utilizando Mineração de Repositório de Software**

Trabalho de Conclusão do Curso de Pós-graduação Latu Sensu apresentado à Universidade Presbiteriana Mackenzie para título de Especialização em Projeto e Desenvolvimento de Sistemas.

Orientador: Prof. Ms. Maurício Aniche

São Paulo

2015

GENEROSO, Leonardo Rodrigues. Padrão evolutivo de arquivos CSS: um estudo em projetos de código aberto utilizando mineração de repositório de software. Trabalho de Conclusão de Curso (Especialização em Projetos e Desenvolvimento de Sistemas) – Centro de Tecnologia da Informação da Universidade Presbiteriana Mackenzie, São Paulo, 2015.

LEONARDO RODRIGUES GENEROSO

PADRÃO EVOLUTIVO DE ARQUIVOS CSS: um estudo em projetos de código aberto utilizando Mineração de Repositório de Software

Trabalho de Conclusão do Curso de Especialização em Projeto e Desenvolvimento de Sistemas apresentado ao Centro de Tecnologia da Informação da Universidade Presbiteriana Mackenzie para obtenção do grau de Pós-graduação Lato Sensu.

Aprovado em __ / __ / __

Prof. Dr. Nome do Orientador (orientador)
Universidade Presbiteriana Mackenzie

Prof. Dr. Nome do Avaliador (avaliador)
Universidade de São Paulo

.

Dedico este trabalho aos meus pais, minha irmã e a todos os amigos que me apoiaram e me deram forças para continuar.

AGRADECIMENTOS

Aos meus pais, Norivaldo Generoso e Maria de Lourdes Rodrigues Generoso por toda dedicação e carinho quem me deram, pois sem eles eu não teria chegado até aqui.

À minha irmã Thais Rodrigues Generoso por todos os momentos juntos.

Aos meus amigos que sempre me apoiaram e me ajudaram ao longo da minha carreira acadêmica e profissional.

Ao meu orientador Prof. Ms. Maurício Aniche por todo o auxílio, apoio e incentivo durante o desenvolvimento desta pesquisa.

À todos aqueles que contribuíram direta ou indiretamente pela conclusão deste trabalho.

"No que diz respeito ao empenho, ao compromisso, ao esforço, à dedicação, não existe meio termo. Ou você faz uma coisa bem feita ou não faz." (Ayrton Senna)

RESUMO

Este projeto tem como objetivo a análise da evolução de arquivos CSS (*Cascading Style Sheet*) em projetos de software de código aberto e a medição do seu padrão evolutivo, abordando aspectos de seu comportamento. Com os benefícios provenientes de sua aplicação, CSS é uma tecnologia muito utilizada em desenvolvimento Web e está presente na camada de *View* em uma arquitetura MVC (*Model-View-Controller*). Porém, garantir a sua escalabilidade e manutenibilidade ao longo do tempo em projetos de software não é algo trivial e uma grande quantidade de esforço pode ser despendida para assegurar a qualidade do código-fonte gerado. Desta forma, torna-se crucial identificar os pontos que são mais propensos a gerar falhas e, a partir de então, criar estratégias para atuar e corrigir esses pontos falhos. Entretanto, efetuar essa identificação não é uma tarefa simples ou rápida. Diversas informações de projeto e desenvolvimento de software são geradas indiretamente através de *commits* e submissões de arquivos no repositório de código, versionamento de código-fonte, lista de discussões, e-mails, entre outros, gerando assim, uma grande massa de dados. Voltada à extração automatizada de informações provenientes dessas fontes, existe a área de MSR (*Mining Software Repositories*). A proposta para este projeto é, através da utilização de mineração de dados em repositórios de software, extrair e analisar algumas das informações geradas durante o processo de desenvolvimento de software em arquivos CSS e inferir informações relevantes sobre os padrões de qualidade do código, os tipos de alterações mais frequentes e quais os impactos que essas alterações podem gerar em outros módulos.

Palavras-chave: Cascading Style Sheets, Mining Software Repositories, Repositório de Software, Código-fonte.

ABSTRACT

This project aims at the analysis of CSS files (Cascading Style Sheet) in open source software projects and measuring its evolution pattern, addressing aspects of their behavior. With the benefits from its implementation, CSS is a technology widely used in Web development and is present in the layer of View in an MVC architecture (Model-View-Controller). However, ensure its scalability and maintainability over time in software projects is not trivial and a lot of effort can be expended to ensure the quality of the their source code. Thus, it is crucial to identify the points that are more likely to generate failures and, from then, create strategies to work and correct these weak points. However, making this identification is not a simple or quick task. Several design information and software development are generated indirectly through committing the source code on repository, versioning source code, mailing list, e-mails, among others, thus generating a large amount of data. Focused on automated extraction of information from these sources, there is the area of Mining Software Repositories (MSR). The proposal for this project is, through the use of mining software repositories data, extract and analyze some information in CSS files, generated during the software development process, and infer relevant information about the code quality standards, more frequent changes types and the impacts that these changes can translate into other modules.

Keywords: Cascading Style Sheets, Mining Software Repositories, Software Repository, Source Code.

LISTA DE ILUSTRAÇÕES

Ilustração 1 - Arquitetura MVC	21
Ilustração 2 - Estrutura da regra.....	23
Ilustração 3 - Estrutura da declaração.....	24
Ilustração 4 - Diagrama geral de uma regra.....	24
Ilustração 5 - Vínculo entre CCS e HTML utilizando o elemento style	26
Ilustração 6 - Renderização da aplicação das regras de estilo	26
Ilustração 7 - Vínculo entre CCS e HTML utilizando o atributo style	27
Ilustração 8 - Vínculo entre CCS e HTML utilizando o elemento link	28
Ilustração 9 - Vínculo entre CCS e HTML utilizando a diretiva @import	28
Ilustração 10 - Exemplo de utilização da diretiva @import.....	29
Ilustração 11 - Seletor de Tipo – Exemplo 1.....	30
Ilustração 12 - Seletor de Tipo – Exemplo 2.....	30
Ilustração 13 - Seletor de Classe – Exemplo 1.....	30
Ilustração 14 - Seletor de Classe – Exemplo 2.....	31
Ilustração 15 - Seletor de ID – Exemplo 1.....	32
Ilustração 16 - Exemplo de CSS Reset.....	33
Ilustração 17 - Passos do Processo KDD	40
Ilustração 18 – Histograma – Seletores de Tipo.....	48
Ilustração 19 – Histograma – Seletores de Classe.....	49

Ilustração 20 – Histograma – Seletores de ID50

LISTA DE TABELAS

Tabela 1 - Projetos Analisados.....44

Tabela 2 - Commits47

LISTA DE ABREVIATURAS, SIGLAS E SÍMBOLOS

CSS	<i>Cascading Style Sheet</i>
HTML	<i>Hyper Text Markup Language</i>
KDD	<i>Knowledge Discovery in Databases</i>
MSR	<i>Mining Software Repositories</i>
MVC	<i>Model-View-Controller</i>
PHP	<i>PHP: Hypertext Preprocessor</i>
UML	<i>Unified Modeling Language</i>

SUMÁRIO

1 INTRODUÇÃO.....	15
1.1 JUSTIFICATIVA	15
1.2 PROBLEMA E QUESTÃO DE PESQUISA.....	16
1.3 OBJETIVOS	16
1.4 METODOLOGIA DO TRABALHO CIENTÍFICO	17
2 DESENVOLVIMENTO WEB	20
3 CASCADING STYLE SHEETS	23
3.2 VÍNCULO ENTRE ARQUIVOS CSS E HTML.....	25
3.2.1 Vínculo utilizando o elemento <i>style</i>	25
3.2.2 Vínculo utilizando o atributo <i>style</i>	27
3.2.3 Vínculo utilizando o elemento <i>link</i>	27
3.2.4 Vínculo utilizando a diretiva <i>@import</i>	28
3.3 SELETORES	29
3.3.1 Seletores de Tipo.....	29
3.3.2 Seletores de Classe	30
3.3.3 Seletores de ID	31
3.4 DESENVOLVIMENTO E MANUTENÇÃO	32
3.4.1 CSS <i>Reset</i>	32
3.4.2 Constantes arbitrárias.....	33
3.4.3 Acoplamento entre um estilo CSS e uma <i>tag</i> HTML.....	34

3.4.4 Uso excessivo de Seletores de ID.....	35
4 DÍVIDA TÉCNICA	36
5 MINERAÇÃO DE DADOS	38
5.1 MINING SOFTWARE REPOSITORIES	40
5.2 METRICMINER	42
6 METODOLOGIA.....	43
6.1 MÉTRICAS.....	45
6.2 IMPLEMENTAÇÃO.....	45
7 RESULTADOS OBTIDOS.....	46
8 CONSIDERAÇÕES FINAIS	51
8.1 TRABALHOS FUTUROS.....	52
REFERÊNCIAS	53

1 INTRODUÇÃO

Com o surgimento do desenvolvimento web, foram criadas diversas tecnologias e metodologias para a criação e desenvolvimento nesta plataforma. Dentre essas tecnologias pode se destacar *Cascading Style Sheets* (CSS).

CSS é uma linguagem para determinar a apresentação de documentos escritos em linguagem de marcação e a sua utilização permite a separação do conteúdo de um documento e a sua respectiva formatação. Um arquivo CSS é um conjunto de Regras que, por sua vez, são compostas por duas partes: o Seletor e a Declaração. O seletor é responsável por definir em quais elementos do documento serão aplicadas as declarações de estilo.

Os Seletores de Tipo, Seletores de Classe e Seletores de ID possuem características específicas e, a maneira como são desenvolvidos e utilizados em um projeto de software pode influenciar na manutenibilidade do sistema e na capacidade de reutilização dos mesmos.

1.1 JUSTIFICATIVA

É possível salientar que com a evolução dos projetos, arquivos CSS e os seus seletores podem ser produzidos em grandes escalas e, caso não sejam adotadas boas práticas de desenvolvimento, pode-se tornar difícil garantir a manutenibilidade e reusabilidade ao longo do tempo.

Por isso, é de extrema importância que sejam identificados pontos nos quais não foram empregadas boas práticas de desenvolvimento, soluções adequadas ou que foram tomadas decisões de design erradas e que podem estar propensas a gerar falhas ou tornar-se dívidas técnicas ao longo do tempo. A partir desse

levantamento, pode-se desenvolver estratégias para corrigir tais pontos, otimizar e padronizar o desenvolvimento dos projetos.

Para o estudo apresentado é utilizada a mineração de repositório de software, que permite extrair informações a partir dos dados gerados ao longo do desenvolvimento de projetos de software.

1.2 PROBLEMA E QUESTÃO DE PESQUISA

O desenvolvimento deste projeto de pesquisa visa responder a seguinte questão: Qual é o padrão evolutivo de arquivos CSS ao longo do tempo de um projeto de software?

1.3 OBJETIVOS

O objetivo desse trabalho é identificar qual é o comportamento evolutivo de arquivos CSS ao longo do tempo de projetos de software e determinar informações relevantes sobre a tendência de evolução dos Seletores de Tipo, Seletores de Classe e Seletores de ID. Desta maneira será possível saber qual tipo de seletor é o mais comumente criado; qual a taxa de criação e remoção destes seletores e qual a quantidade de manutenção que os arquivos CSS demandam.

1.4 METODOLOGIA DO TRABALHO CIENTÍFICO

O método científico é o conjunto das atividades sistemáticas e racionais que, com maior segurança e economia, permite alcançar o objetivo - conhecimentos válidos e verdadeiros -, traçando o caminho a ser seguido, detectando erros e auxiliando as decisões do cientista (MARCONI; LAKATOS, 2003).

Com o passar do tempo, muitas modificações foram feitas nos métodos existentes, inclusive surgiram outros novos (MARCONI; LAKATOS, 2003).

O método científico é a teoria da investigação. Esta alcança seus objetivos, de forma científica, quando cumpre ou se propõe a cumprir as seguintes etapas: (MARCONI; LAKATOS, 2003)

- i) Descobrimento do problema ou lacuna num conjunto de conhecimentos. Se o problema não estiver enunciado com clareza, passa-se à etapa seguinte; se o estiver, passa-se à subsequente;
- ii) Colocação precisa do problema, ou ainda a recolocação de um velho problema, à luz de novos conhecimentos (empíricos ou teóricos, substantivos ou metodológicos);
- iii) Procura de conhecimentos ou instrumentos relevantes ao problema (por exemplo, dados empíricos, teorias, aparelhos de medição, técnicas de cálculo ou de medição). Ou seja, exame do conhecido para tentar resolver o problema;
- iv) Tentativa de solução do problema com auxílio dos meios identificados. Se a tentativa resultar inútil, passa-se para a etapa seguinte; em caso contrário, à subsequente;
- v) Invenção de novas idéias (hipóteses, teorias ou técnicas) ou produção de novos dados empíricos que prometam resolver o problema;
- vi) Obtenção de uma solução (exata ou aproximada) do problema com auxílio do instrumental conceitual ou empírico disponível;
- vii) Investigação das consequências da solução obtida. Em se tratando de uma teoria, é a busca de prognósticos que possam ser feitos com seu

auxílio. Em se tratando de novos dados, é o exame das consequências que possam ter para as teorias relevantes;

- viii) Prova (comprovação) da solução: confronto da solução com a totalidade das teorias e da informação empírica pertinente. Se o resultado é satisfatório, a pesquisa é dada como concluída, até novo aviso. Do contrário, passa-se para a etapa seguinte;
- ix) Correção das hipóteses, teorias, procedimentos ou dados empregados na obtenção da solução incorreta. Esse é, naturalmente, o começo de um novo ciclo de investigação (BUNGE, 1980).

De todos os métodos científicos citados pelas autoras MARCONI e LAKATOS (2003), é interessante ressaltar o método indutivo, pois é o método ao qual esse trabalho se enquadra.

Indução é um processo mental por intermédio do qual, partindo de dados particulares, suficientemente constatados, infere-se uma verdade geral ou universal, não contida nas partes examinadas. Portanto, o objetivo dos argumentos indutivos é levar a conclusões cujo conteúdo é muito mais amplo do que o das premissas nas quais se basearam (MARCONI; LAKATOS, 2003).

Devemos considerar três elementos fundamentais para toda indução, isto é, a indução realiza-se em três etapas (fases):

- i) Observação dos fenômenos - nessa etapa observamos os fatos ou fenômenos e os analisamos, com a finalidade de descobrir as causas de sua manifestação;
- ii) Descoberta da relação entre eles - na segunda etapa procuramos por intermédio da comparação, aproximar os fatos ou fenômenos, com a finalidade de descobrir a relação constante existente entre eles;
- iii) Generalização da relação - nessa última etapa generalizamos a relação encontrada na precedente, entre os fenômenos e fatos semelhantes, muitos dos quais ainda não observamos (e muitos inclusive inobserváveis) (MARCONI; LAKATOS, 2003).

O presente trabalho trata-se de uma pesquisa pura, partindo do método indutivo, com o objetivo de conhecer o comportamento dos arquivos CSS ao longo do tempo de projetos de software. As ações que podem ser tomadas a partir do prévio conhecimento da tendência comportamental destes arquivos irá depender de fatores como a complexidade do projeto, sua duração, tamanho da equipe, entre outros.

Para a realização desta pesquisa, foram definidas métricas para que fosse possível calcular o padrão evolutivo de cada um dos seletores presentes em um arquivo CSS, de forma que este trabalho possui um caráter de pesquisa quantitativa.

Como um trabalho de pesquisa descritiva, foi analisado o comportamento dos Seletores de Tipo, Seletores de Classe e Seletores de ID, com o intuito de descrever suas características comportamentais e, então, tentar estabelecer sua relação com a capacidade de reusabilidade e necessidade de manutenção.

Por fim, este trabalho trata-se de um estudo de caso, onde foram selecionados e analisados 22 projetos web de código aberto. A lista de projetos, contendo informações sobre o seu tamanho, complexidade e quantidade de manutenção em arquivos CSS é apresentada no capítulo 6 METODOLOGIA. Neste capítulo são descritas, também, a abordagem e as métricas definidas para a análise e medição proposta.

2 DESENVOLVIMENTO WEB

Desde o surgimento do desenvolvimento web, diversas tecnologias, metodologias e arquiteturas foram criadas, e este número ainda é crescente. Uma das arquiteturas mais difundidas e utilizadas para desenvolvimento web na atualidade é a arquitetura MVC (*Model-View-Controller*) (OLIVEIRA Jr; FORTES, 2010).

A arquitetura MVC foi desenvolvida por Trygve Reenskaug em 1978/79, quando era cientista da Xerox PARC (*Palo Alto Research Laboratory*). REENSKAUG afirma que a arquitetura MVC foi criada como uma solução para o problema geral de dar ao usuário o controle sobre as informações e como acessá-las a partir de múltiplas perspectivas (REENSKAUG, 2007).

Inicialmente, o termo utilizado para esta arquitetura era *Model-View-Editor*, apresentado na publicação de 12 de maio de 1979, que continha as definições iniciais da arquitetura e exemplos práticos. Mas em 10 de dezembro de 1979, em outra publicação do autor, o termo foi alterado para *Model-View-Controller* (REENSKAUG, 1979).

Como o nome já pressupõe, o padrão arquitetural MVC é composto por três componentes: *Model*, *View* e *Controller* onde cada um desses componentes possui responsabilidades definidas (REENSKAUG, 1979).

O componente *Model* é responsável pelo encapsulamento da funcionalidade, por manter os dados, estados e a lógica de uma determinada abstração. O componente *Model* é independente de qualquer representação de saída de dados específicas ou comportamentos de entrada de dados. REENSKAUG define *Model* como a representação de uma abstração sob a forma de dados em um sistema computacional (REENSKAUG, 1979).

O componente *View* destina-se à apresentação das informações do sistema para o usuário. As informações são obtidas a partir do componente *Model* e podem existir múltiplas *Views* para um *Model*. REENSKAUG diz que para um determinado

Model existe uma ou mais *Views* vinculadas, capazes de efetuar uma representação do componente de modelo (REENSKAUG, 1979).

Por sua vez, o componente *Controller* possui o objetivo de manipular os *inputs* no sistema. Cada *View* tem associada a ela um componente *Controller*, que recebem as entradas e as traduzem em requisições de serviço para o *Model*. REENSKAUG afirma que o *Controller* é o elo entre o usuário e o sistema, provendo formas de input de dados (REENSKAUG, 1979).

Na Ilustração 1 é apresentada a estrutura dos componentes e o padrão de interação da arquitetura MVC.

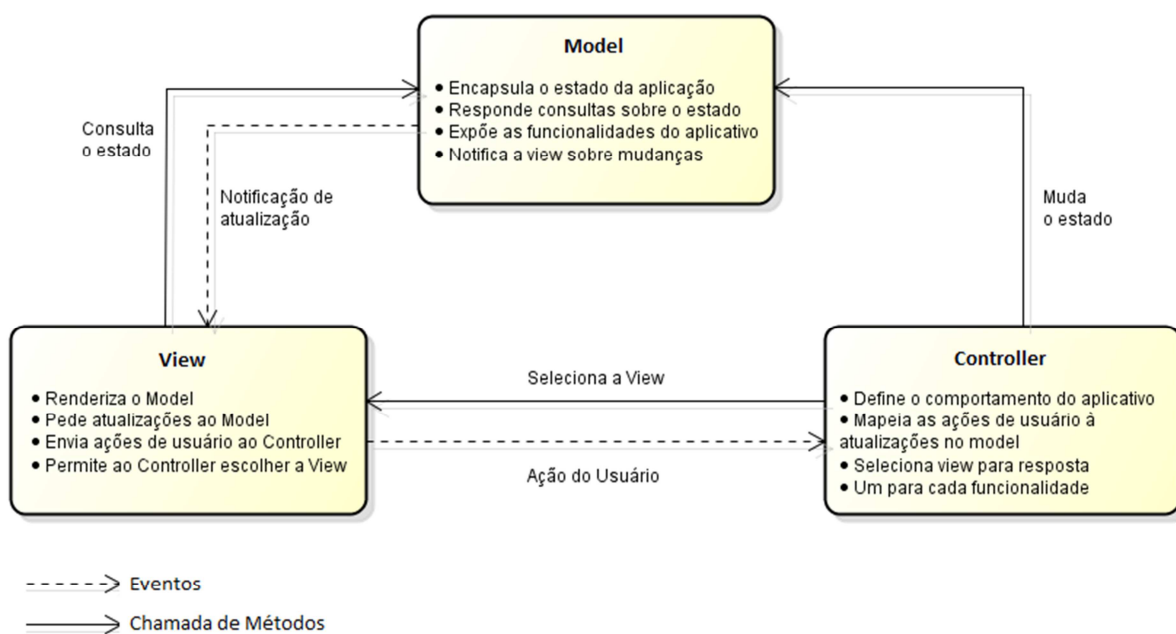


Ilustração 1 - Arquitetura MVC

Fonte: ECKSTEIN (2007)

Dentre as tecnologias presentes no desenvolvimento web, é possível destacar o HTML (*Hyper Text Markup Language*), linguagem de marcação responsável pela definição de vários componentes que compõem um documento da internet (LONGMAN, 1998); PHP (*PHP: Hypertext Preprocessor*, inicialmente chamada de

Personal Home Page), que é uma linguagem multiplataforma originalmente criada para o desenvolvimento de aplicações atuantes e processadas no servidor (PHP); e CSS (*Cascading Style Sheets*), que é utilizada para a definição da visualização de documentos escritos em linguagem de marcação, como por exemplo HTML (W3C). A utilização do CSS permite que, em desenvolvimentos web, exista a separação de conteúdo de um documento e a sua formatação (W3C).

Dentre as tecnologias web citadas, CSS é executada dentro da camada *View* da arquitetura MVC e é o foco de estudo deste trabalho. Serão abordados aspectos de seu comportamento, a forma de utilização e sua relação com documentos HTML, descrição de alguns problemas que arquivos CSS podem conter e as dificuldades de manutenção existentes. Por fim, será efetuada a análise para identificação do padrão evolutivo em projetos de software.

3 CASCADING STYLE SHEETS

Cascading Style Sheet é um mecanismo simples para adicionar estilos (e.g. fontes, cores, espaçamento) em documentos Web (W3C, 2014).

Segundo os autores LIE e BOS, uma folha de estilo (*Style Sheet*) é um conjunto de uma ou mais regras que se aplicam a um documento HTML.

Cada documento pode estar atrelado a uma ou mais folhas de estilo que afetarão a sua apresentação, residentes em diferentes locais, como por exemplo o navegador, as folhas de estilos dos autores do documento e, possivelmente, as preferências de estilos do usuário (LIE; BOS, 1999).

Para o gerenciamento de todas essas folhas de estilo existe uma funcionalidade fundamental do CSS, o efeito cascata. Este efeito faz com que todas as diferentes folhas de estilos sejam consideradas e aplicadas como se estivessem em série (LIE; BOS, 1999).

3.1 ESTRUTURA DAS REGRAS

Uma regra em um arquivo CSS é composta por duas partes: *Selector* (Seletor) e *Declaration* (Declaração) (LIE; BOS, 1999).

Na Ilustração 2 - Estrutura da regra, é apresentado um exemplo desta estrutura.

```
H1 { color: green }
```

O diagrama mostra a estrutura de uma regra CSS. A linha de código "H1 { color: green }" é exibida. Abaixo dela, duas linhas verticais indicam a divisão da regra. A primeira linha, alinhada com "H1", é rotulada como "Selector". A segunda linha, alinhada com "{ color: green }", é rotulada como "Declaration".

Ilustração 2 - Estrutura da regra

Fonte: LIE; BOS (1999)

O Seletor atua como um *link* entre uma marcação HTML do documento e a estilização. Ele é responsável por definir em quais elementos do documento serão aplicadas as declarações de estilo. Já a Declaração é responsável por definir o efeito de estilização que será aplicado (LIE; BOS, 1999).

Por sua vez, uma Declaração também é constituída por duas partes: *Property* (Propriedade) e *Value* (Valor), estas separadas por dois pontos (LIE; BOS, 1999).

Na Ilustração 3 - Estrutura da declaração, é apresentado um exemplo desta estrutura.



Ilustração 3 - Estrutura da declaração

Fonte: LIE; BOS (1999)

A Propriedade é cada uma das qualidades ou características próprias de um determinado item. O Valor é a definição para o item, o comportamento que ele irá assumir (LIE; BOS, 1999).

Na Ilustração 4 - Diagrama geral de uma regra, é apresentado um exemplo de uma regra, com as partes que a compõe.

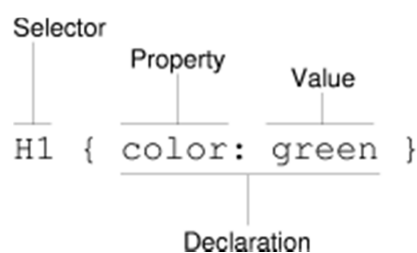


Ilustração 4 - Diagrama geral de uma regra

Fonte: LIE; BOS (1999)

3.2 VÍNCULO ENTRE ARQUIVOS CSS E HTML

Para a utilização de uma folha de estilo em um arquivo HTML, deve ser criado um vínculo entre eles. Este vínculo pode ser feito de quatro maneiras distintas, sendo elas: (LIE; BOS, 1999)

- Utilização do elemento *style* dentro do arquivo HTML, aplicando estilo à todo o documento;
- Utilização do atributo *style* de elementos HTML, aplicando o estilo individualmente;
- Criação de um *link* do documento HTML para um arquivo CSS externo através do elemento *link*;
- Importação de um arquivo CSS através da diretiva CSS *@import*.

3.2.1 Vínculo utilizando o elemento *style*

Nesta forma de vínculo, a folha de estilos é inserida no documento HTML através da utilização do elemento *style*. Todas as informações da folha de estilo ficam dentro deste elemento, que é adicionado ao topo do documento HTML (LIE; BOS, 1999).

Na Ilustração 5 é exibido um exemplo de vínculo entre folhas de estilos e documentos HTML através da utilização do elemento *style*.

```

<HTML>
  <TITLE>Bach's home page</TITLE>

  <STYLE>
    H1, H2 { color: green }
  </STYLE>

  <BODY>
    <H1>Bach's home page</H1>
    <P>Johann Sebastian Bach was a prolific
      composer. Among his works are:
    <UL>

      <LI>the Goldberg Variations
      <LI>the Brandenburg Concertos
      <LI>the Christmas Oratorio
    </UL>
    <H2>Historical perspective</H2>
    <P>Bach composed in what has been referred to as
      the Baroque period.
  </BODY>

</HTML>

```

Ilustração 5 - Vínculo entre CCS e HTML utilizando o elemento *style*

Fonte: LIE; BOS (1999)

O resultado do código-fonte presente na Ilustração 5 pode ser visualizado na Ilustração 6.

Bach's home page

Johann Sebastian Bach was a prolific composer. Among his works are:

- the Goldberg Variations
- the Brandenburg Concertos
- the Christmas Oratorio

Historical perspective

Bach composed in what has been referred to as the Baroque period.

Ilustração 6 - Renderização da aplicação das regras de estilo

Fonte: LIE; BOS (1999)

3.2.2 Vínculo utilizando o atributo *style*

A linguagem HTML fornece um atributo *style* para a maioria dos elementos existentes, permitindo a definição de estilos para cada elemento em específico (ÇELIK; BOS; ATTINASI, 2015).

Na Ilustração 7 - Vínculo entre CCS e HTML utilizando o atributo *style* é apresentado um exemplo do vínculo utilizando o atributo *style*.

```
<p style="color: #090; line-height: 1.2">...</p>
```

Ilustração 7 - Vínculo entre CCS e HTML utilizando o atributo *style*

Fonte: ÇELIK; ETERMAD (2015)

3.2.3 Vínculo utilizando o elemento *link*

Este elemento define um *link* para outro arquivo. Este elemento só pode ser inserido na seção *head* de um documento HTML, embora possa aparecer mais do que uma vez (W3.ORG, 2015).

Quando o elemento *link* é utilizado para vincular uma folha de estilo à um documento, o atributo *type* especifica o padrão de linguagem da folha de estilo e o atributo *href* determina a localização da folha de estilo (W3.ORG, 2015).

Na Ilustração 8 - Vínculo entre CCS e HTML utilizando o elemento *link* é apresentado um exemplo do vínculo utilizando o elemento *link*.

```

<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>Example with link elements</title>
    <link rel="stylesheet" href="common.css"/>
    <link rel="stylesheet" href="default.css" title="Default style"/>
    <link rel="alternate stylesheet" href="alt.css" title="Alternative style"/>
    <link rel="stylesheet" href="single-col.css" media="all and (max-width: 30em)"/>
  </head>
  <body>
    ...
  </body>
</html>

```

Ilustração 8 - Vínculo entre CCS e HTML utilizando o elemento *link*

Fonte: W3.ORG (2015)

3.2.4 Vínculo utilizando a diretiva *@import*

A diretiva *@import* deve preceder todas as outras regras em uma folha de estilo. Esta diretiva por ser dependente de um formato específico de mídia, neste caso, cada um dos formatos deve ser especificado e, somente nesses formatos, serão aplicadas as definições de estilo. Caso não exista uma definição de formato de mídia, a importação do estilo será aplicada para qualquer mídia (LIE; ATKINS, 2013).

Na Ilustração 9 é apresentada a sintaxe de utilização da diretiva *@import* e na Ilustração 10 é apresentado um exemplo de utilização da diretiva.

```

<import-rule> = @import [ <url> | <string> ] <media-query-list>? ;

```

Ilustração 9 - Vínculo entre CCS e HTML utilizando a diretiva *@import*

Fonte: LIE; ATKINS (2013)

```
@import url("fineprint.css") print;  
@import url("bluish.css") projection, tv;  
@import url("narrow.css") handheld and (max-width: 400px);
```

Ilustração 10 - Exemplo de utilização da diretiva *@import*

Fonte: LIE; ATKINS (2013)

3.3 SELETORES

Seletores são estruturas em um arquivo CSS utilizadas para efetuar o vínculo entre as definições de um estilo à elementos do documento HTML (ÇELIK et al., 2011).

3.3.1 Seletores de Tipo

Um Seletor de Tipo representa uma instância de um determinado tipo de elemento em um documento HTML (ÇELIK et al., 2011).

Este seletor é utilizado para aplicar estilos em todos os elementos de um mesmo tipo (e.g. <p>, <h3>, <div>).

O nome deste seletor deve ser o nome do elemento no qual deseja-se aplicar o estilo.

Na Ilustração 11 e na Ilustração 12 são apresentados exemplos de Seletor de Tipo.

```
p {  
  text-align: center;  
  color: red;  
}
```

Ilustração 11 - Seletor de Tipo – Exemplo 1

Fonte: Autor

```
h3 {  
  font-family: "Comic Sans MS";  
}
```

Ilustração 12 - Seletor de Tipo – Exemplo 2

Fonte: Autor

3.3.2 Seletores de Classe

O Seletor de Classe é aplicado para qualquer elemento do documento HTML que contenha o atributo *class* definido com o respectivo nome da classe.

O nome deste tipo de seletor deve ser composto pelo caractere “.”, seguido pelo nome da classe.

Na Ilustração 13 é apresentado um exemplo de um Seletor de Classe.

```
.center {  
  text-align: center;  
  color: red;  
}
```

Ilustração 13 - Seletor de Classe – Exemplo 1

Fonte: Autor

É possível restringir a aplicação do Seletor de Classe para elementos específicos do documento HTML adicionando o nome do elemento antes do caractere “.”.

Na Ilustração 14 é apresentado um exemplo de um Seletor de Classe que somente pode ser utilizado por elementos <p>.

```
p.center {  
    text-align: center;  
    color: red;  
}
```

Ilustração 14 - Seletor de Classe – Exemplo 2

Fonte: Autor

3.3.3 Seletores de ID

O Seletor de ID é aplicado para qualquer elemento do documento HTML que contenha o atributo *id* definido com o respectivo nome do seletor. Entretanto, o ID deve ser único no documento HTML, podendo existir apenas um elemento de um determinado ID.

O nome deste tipo de seletor deve ser composto pelo caractere “#”, seguido pelo nome do ID.

Na Ilustração 15 é apresentado um exemplo de um Seletor de ID.


```
#parag1 {  
    text-align: center;  
    color: red;  
}
```

Ilustração 15 - Seletor de ID – Exemplo 1

Fonte: Autor

3.4 DESENVOLVIMENTO E MANUTENÇÃO

Por possuir uma estrutura simples e agregar os benefícios provenientes da separação entre conteúdo e estilo, CSS é muito utilizado em desenvolvimento Web. Porém, apesar de sua fácil aplicação, alguns aspectos de sua utilização devem ser vistos e boas práticas de desenvolvimento devem ser adotadas a fim de garantir a sua escalabilidade e manutenibilidade.

A seguir, são apresentados alguns pontos que podem gerar problemas no desenvolvimento e manutenção de arquivos CSS.

3.4.1 CSS Reset

Os navegadores web possuem um estilo padrão para os elementos em um documento HTML. Este estilo padrão é aplicado a um elemento que não possua nenhuma definição através da utilização de CSS.

Porém, é possível que em alguns casos, navegadores distintos possuam diferentes definições de estilos para um mesmo elemento, gerando assim diferenças em sua visualização.

Para tentar garantir um padrão de exibição, existe o *CSS Reset*. Ele consiste em limpar as definições dos elementos. Na Ilustração 16 é apresentado um exemplo de *CCS Reset*.

```
* { margin:0; padding:0; }
```

Ilustração 16 - Exemplo de *CSS Reset*

Fonte: Autor

Um problema na utilização de *CSS Reset* é que se ele não for aplicado de maneira adequada e somente nos elementos necessários, acaba gerando um cenário onde muito código fonte é escrito apenas para que um determinado elemento volte a ter um comportamento semelhante ao inicial, antes do *Reset*.

3.4.2 Constantes arbitrárias

A utilização de valores arbitrários na definição de margem, largura, altura e outras propriedades numéricas podem gerar um *CSS* pouco reaproveitável e diminuir a facilidade de manutenção.

Em um cenário onde temos uma página com 940px de largura, e gostaríamos de dividi-la em 4 colunas iguais, poderíamos simplesmente definir a largura da coluna com a expressão:

width:235px

Entretanto, essa abordagem dificulta a manutenção ao longo do tempo, pois a largura da página pode mudar e, além disso, o desenvolvedor não irá saber a origem lógica desse valor.

Ao invés de utilizar valores arbitrários, uma melhor abordagem nessa situação seria o emprego de percentuais, desta forma, a expressão ficaria:

width:25%

3.4.3 Acoplamento entre um estilo CSS e uma *tag* HTML

A utilização de *tags* do HTML diretamente no CSS aumenta o acoplamento entre eles, de forma que o estilo fica dependente da estrutura HTML.

Nestes casos, uma alteração na estrutura HTML fará com que o estilo deixe de funcionar, gerando uma necessidade de manutenção no arquivo CSS.

Em um cenário onde temos um *menu* com uma lista de itens, poderia ser utilizada a seguinte expressão para a definição de um estilo no arquivos CSS:

.menu li { ... }

Porém, este estilo ficará acoplado a atual estrutura do arquivo HTML. Uma melhor abordagem para este cenário seria a criação de uma nova classe, contendo as definições de estilo necessárias:

.menu-item { ... }

Com a existência da nova classe, ela pode ser aplicada na *tag* do arquivo HTML.

3.4.4 Uso excessivo de Seletores de ID

Seletores de ID só podem ser vinculados à um elemento no documento HTML, desta forma, tornam-se muito inflexíveis e sua reutilização é ínfima.

É preferível a utilização de Seletores de Classe, pois podem ser utilizados em mais de um elemento, aumentando assim o reaproveitamento do código escrito para os estilos no CSS.

4 DÍVIDA TÉCNICA

Dívida técnica é a dívida que resulta quando você consciente ou inconscientemente toma decisões de design erradas ou não ideais (SURYANARAYANA; SAMARTHYAM; SHARMA, 2014).

Dívida técnica é uma metáfora cunhada por Ward Cunningham em 1992, que faz uma analogia à dívida financeira. Quando uma pessoa toma um empréstimo (ou usa seu cartão de crédito), ela adquire uma dívida. Se a pessoa paga regularmente as parcelas, a dívida criada é paga e não gera consequências futuras. No entanto, se a pessoa não pagar esta dívida, uma penalidade na forma de juros é aplicável e é acrescentada ao valor total toda vez que o pagamento não é efetuado. Caso a pessoa não seja capaz de pagar a dívida por um longo tempo, os juros acumulados pode fazer a dívida crescer substancialmente a ponto da pessoa ter que declarar falência (SURYANARAYANA; SAMARTHYAM; SHARMA, 2014).

Da mesma forma, quando um desenvolvedor opta por uma solução rápida ao invés de uma solução bem adequada e bem projetada, ele cria uma dívida técnica. Não há problema se o desenvolvedor pagar a dívida a tempo. Entretanto, se o desenvolvedor não pagar ou se esquecer da dívida criada, os juros são acumulados, assim como na dívida financeira. A dívida pode continuar a aumentar ao longo do tempo com cada mudança no software, e quanto mais tarde for, mais cara será a dívida a pagar. Se a dívida não for paga por completo, eventualmente, pode tornar-se tão grande a ponto de ser extremamente difícil efetuar manutenções no software. Em casos extremos, a dívida chega a um determinado ponto que não pode mais ser paga e o produto tem de ser abandonado. Tal situação é chamada de falência técnica (SURYANARAYANA; SAMARTHYAM; SHARMA, 2014).

Além dos desafios técnicos, a dívida técnica também impacta na moral e motivação da equipe de desenvolvimento. Com o aumento da dívida técnica, torna-se difícil a manutenção do software e a equipe envolvida com o desenvolvimento pode começar a se sentir frustrada. Essa frustração ainda pode ser agravada porque a alternativa - ou seja, pagar toda a dívida técnica - não é uma tarefa trivial que pode

ser realizada de forma simples ou rápida (SURYANARAYANA; SAMARTHYAM; SHARMA, 2014).

Existem várias fontes de dívidas técnicas, entre elas:

- Dívida em código fonte: constituem falhas estruturais e estilo de codificação inconsistente.
- Dívida em design: composta por arquitetura e design inconsistentes.
- Dívida em teste: falta de testes, cobertura inadequada dos testes, ou teste mal definidos.
- Dívida em documentação: falta de documentação ou documentação incompleta, ou documentação desatualizada (SURYANARAYANA; SAMARTHYAM; SHARMA, 2014).

Segundo os autores NAGAPPAN, BALL e ZELLER, uma grande quantidade de esforço é consumida durante o processo de desenvolvimento de software para garantir a sua qualidade e, desta forma, é necessário identificar as partes do software que são mais propensas a possuir falhas. Para tal, a tecnologia de mineração de dados é utilizada nos repositórios de software a fim de mapear os problemas existentes (NAGAPPAN; BALL; ZELLER, 2006).

5 MINERAÇÃO DE DADOS

Os avanços tecnológicos em poder de processamento, capacidade de armazenamento e interconectividade de tecnologias computacionais vêm criando uma quantidade de dados digitais sem precedentes (CHAKRABARTI et al., 2006).

Mineração de dados é um campo interdisciplinar, abrangendo inteligência artificial, aprendizado de máquina, estatística e sistema de banco de dados. O principal objetivo da mineração de dados é extrair conhecimento e informações a partir de uma massa de dados e agrupá-los em uma estrutura compreensível (CHAKRABARTI et al., 2006).

O desenvolvimento de métodos e técnicas para a mineração de dados é o objetivo do *Knowledge Discovery in Databases* (KDD) (descoberta de conhecimento em base de dados). O KDD visa mapear dados de baixo-nível, tipicamente muito volumosos e de difícil compreensão, em formas mais compactas, mais abstratas ou mais práticas (FAYYARD; PIATETSKY-SHAPIRO; SMYTH, 1996).

O processo de KDD é interativo e iterativo, composto dos seguintes passos:

- O primeiro passo consiste em compreender o domínio da aplicação, obter o conhecimento prévio relevante e identificar o objetivo do processo de KDD do ponto de vista do cliente.
- O segundo passo visa selecionar um conjunto de dados, ou simplesmente um subconjunto de variáveis que serão analisadas.
- O terceiro passo é o pré-processamento e tratamento dos dados, removendo possíveis ruídos e criando estratégias para lidar com eventuais situações onde algumas informações são inexistentes.
- O quarto passo é a redução de dados e projeção, encontrando recursos úteis para representar os dados dependendo do objetivo. Com a redução da dimensionalidade ou mesmo a sua transformação, o número efetivo de variáveis em consideração pode ser reduzido ou representações invariantes dos dados podem ser encontradas.

- O quinto passo busca mapear os objetivos do processo KDD, levantados no passo 1, para um método específico de mineração de dados, como por exemplo, compactação, classificação, regressão, agrupamento, entre outros.
- O sexto passo é a análise exploratória. É efetuada a escolha do algoritmo de prospecção de dados e os métodos que serão utilizados para a geração de padrões de dados.
- O sétimo passo é a mineração dos dados, buscando por padrões de interesse em uma forma de representação particular ou um conjunto de tais representações.
- O oitavo passo foca na interpretação dos padrões minerados, possivelmente retornando a qualquer um dos passos entre 1 e 7 para outra iteração.
- O nono passo, por sua vez, consiste em atuar sobre o conhecimento obtido. Usando o conhecimento diretamente, incorporando o conhecimento em outros sistemas para ações futuras, ou simplesmente documentando-o e reportando-o para as partes interessadas. Este processo inclui também a verificação e resolução de eventuais conflitos com eventuais conhecimentos previamente extraídos.

O processo pode envolver diversas iterações e loops em quaisquer passos (FAYYARD; PIATETSKY-SHAPIRO; SMYTH, 1996).

Na Ilustração 17 é apresentado o fluxo básico dos passos do processo KDD.

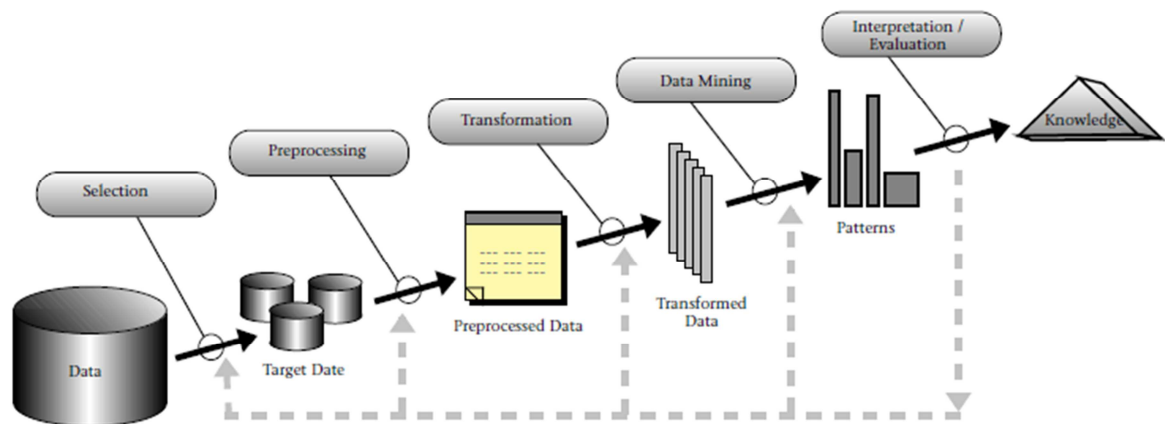


Ilustração 17 - Passos do Processo KDD

Fonte: FAYYARD; PIATETSKY-SHAPIO; SMYTH (1996)

5.1 MINING SOFTWARE REPOSITORIES

Durante o desenvolvimento de um software, programadores rotineiramente produzem grandes quantidades de dados, os quais podem ser acessados e analisados automaticamente. Dentre as possíveis fontes de dados estão:

- i) O código fonte do software, que é um dos principais artefatos, no qual podem ser extraídas informações de classes, componentes, entre outros, podendo assim mapear o relacionamento entre elas e a forma como elas evoluem ao longo do projeto;
- ii) *Logs* e *profiles* da execução, que contém informações sobre as partes do sistema que são frequentemente utilizadas e as partes que são menos acessadas;
- iii) Documentos de requisitos, diagramas UML (*Unified Modeling Language*) e outras documentações arquiteturais do software fornecem informações sobre como o software está construído;

- iv) Versionamento dos arquivos do software, que fornecem informações sobre as alterações efetuadas no produto, incluindo quem, quando, onde e porque foram feitas;
- v) Repositório de *bugs*, o qual fornece informações sobre os problemas que já ocorreram no sistema e o seu histórico de tratamento;
- vi) Dados gerados do relacionamento entre colaboradores, tais como e-mails, mensagens e qualquer outra forma de troca de conhecimento, no qual é possível obter informações sobre nível de contribuição pessoal no projeto. (HERZIG; ZELLER, 2011).

Mining Software Repositories (MSR) visa à análise de informações geradas em projetos de desenvolvimento de software, como repositórios de código-fonte, repositórios de *bug*, controle de versão, listas de discussão, e-mails, entre outros, a fim de mapear e descobrir informações relevantes sobre o padrão evolutivo no desenvolvimento do software (HERZIG; ZELLER, 2011).

Os autores HERZIG e ZELLER definem “*mining software archives*” (mineração de arquivos de software) como um processo para “obter uma gama de informações iniciais” a partir da extração de informações dos repositórios de software. E, ainda, definem “*data sources*” (fontes de dados) como artefatos baseados em código-fonte, requisitos de sistema, arquivos de versão, entre outros. Segundo os autores, esses artefatos são imparciais, pois eles registram as mudanças, problemas e outros eventos no momento em que ocorrem. Mas por outro lado, também possuem ruídos e podem ser incompletos e, desta forma, alguns passos especiais de preparação são necessários antes da sua análise (HERZIG; ZELLER, 2011).

5.2 METRICMINER

Para análise do padrão evolutivo de arquivos CSS em projetos de software, este trabalho utiliza o *MetricMiner*, que é responsável pela mineração dos repositórios de software.

O *MetricMiner* é uma ferramenta Web criada a partir do *rEvolution* que, por sua vez, é uma aplicação de linha de comando que extrai dados de um repositório local e persiste em banco de dados relacional. Todo o código e o histórico de desenvolvimento do *MetricMiner* se encontra hospedado no repositório online *github*: <http://github.com/metricminer/metricminer>. O código do *rEvolution* pode ser encontrado também no *github*: <http://github.com/mauricioaniche/rEvolution> (SOKOL, 2012).

6 METODOLOGIA

O desenvolvimento deste projeto consiste em responder a seguinte questão de pesquisa:

Qual é o padrão evolutivo de arquivos CSS ao longo do tempo de um projeto de software?

Com o objetivo de responder a esta questão, foram utilizadas técnicas de mineração de repositório de software e, assim, implementada uma ferramenta que faz uso do *MetricMiner* para obter informações de cada um dos *commits* realizados nos repositórios.

A ferramenta implementada efetua uma varredura em cada um dos *commits* efetuados nos projetos e, para cada um deles, analisa a estrutura de todos os arquivos CSS encontrados, aplicando as métricas desenvolvidas para responder a questão de pesquisa.

A métrica definida foi analisar qual o padrão de evolução dos seletores existentes nos arquivos CSS, determinando qual tipo de seletor é mais comumente criado: Seletores de Tipo, Seletores de Classe ou Seletores de ID.

Como fonte de estudo para este projeto, foram selecionados 22 projetos web de código aberto. A lista de projetos, contendo a quantidade de arquivos CSS, a quantidade total de *commits* e a quantidade de *commits* envolvendo um ou mais arquivos CSS é apresentada na Tabela 1 - Projetos Analisados.

Tabela 1 - Projetos Analisados

Projeto	Arquivos CSS	Total de Commits	Commits Envolvendo Arquivos CSS	Porcentagem de Commits de Arquivos CSS
Atlas	250	4780	344	7,20 %
Donor-Connect	7	207	24	11,59 %
eeg-database	37	1612	102	6,33 %
Ehour	82	4267	363	8,51 %
Esporx	62	346	57	16,47 %
ff-core	9	224	21	9,38 %
Fórum	10	494	80	16,19 %
Gitblit	53	2730	176	6,45 %
graylog2-web-interface	65	3508	628	17,90 %
Head	198	12922	351	2,72 %
motech-whp	38	2462	190	7,72 %
Mystamps	17	1344	30	2,23 %
open-lmis	71	6822	449	6,58 %
openmrs-com	21	898	33	3,67 %
org.qibud.project	44	111	8	7,21 %
PhenotypeArchive	111	8552	437	5,11 %
Springtrader	328	1067	147	13,78 %
TAMA-Web	36	2722	114	4,19 %
Toolkit	16	301	30	9,97 %
TracksAnalytics	1	308	1	0,32 %
WebCash	7	221	10	4,52 %
zanata-server	232	11336	898	7,92 %

Fonte: Autor

6.1 MÉTRICAS

Para a análise quantitativa dos tipos de seletores presentes nos arquivos CSS, cada um deles, Seletores de Tipo, Seletores de Classe e Seletores de ID, são identificados dentro dos arquivos CSS e contabilizados.

Para a identificação dos tipos de seletores, foi assumido o seguinte conjunto de regras:

- i) Se o nome do seletor possuir o caractere “.”, este seletor será identificado como um Seletor de Classe;
- ii) Se o nome seletor possuir o caractere “#”, este seletor será identificado como um Seletor de ID;
- iii) Se o nome do seletor não possuir nenhum dos caracteres supracitados, este seletor será identificado como um Seletor de Tipo.

6.2 IMPLEMENTAÇÃO

Para a implementação da métrica definida, foi desenvolvido utilizando a linguagem Java, um analisador sintático simples. Este analisador é responsável por interpretar a estrutura dos arquivos CSS e identificar cada um dos seletores existentes e seus nomes.

Com a obtenção dos seletores, é aplicado o conjunto de regras definido e assim identificado o tipo de seletor.

7 RESULTADOS OBTIDOS

Na Tabela 1 - Projetos Analisados, é apresentado, também, a porcentagem dos *commits* que envolveram alterações em arquivos CSS. Esta porcentagem variou de 0,32% a 17,90%, com uma média de 8%. Este valor apresenta uma taxa relativamente baixa de necessidade de manutenção nos arquivos CSS ao longo do projeto.

Na Tabela 2 - *Commits*, são apresentados os resultados obtidos da análise geral dos projetos. Foi avaliada a evolução de cada um dos arquivos CSS existentes em cada projeto comparando o seu estado no primeiro e no último *commit* e, para os Seletores de Tipo, Seletores de Classe e Seletores de ID, são apresentados os seguintes valores:

- i) Média da quantidade de cada tipo de seletor no primeiro *commit* dos arquivos;
- ii) Média da quantidade de cada tipo de seletor no último *commit* dos arquivos;
- iii) Média da porcentagem da quantidade de cada tipo de seletor no primeiro *commit* dos arquivos;
- iv) Média da porcentagem da quantidade de cada tipo de seletor no último *commit* dos arquivos;
- v) Diferença entre a média da quantidade de cada tipo de seletor no primeiro e no último *commit* dos arquivos;
- vi) Porcentagem equivalente à diferença da média entre o primeiro e o último *commit* dos arquivos.

Com estas informações é possível verificar que, dentre os tipos de seletores, o Seletor de Classe foi o que apresentou a maior quantidade, enquanto o Seletor de ID teve a menor incidência.

É possível identificar, também, um decréscimo na quantidade de cada um dos tipos de seletores.

Tabela 2 - *Commits*

Seletor	Média do Primeiro <i>Commit</i>	Média do Último <i>Commit</i>	% no Primeiro <i>Commit</i>	% no Último <i>Commit</i>	Diferença	% da Diferença
Tipo	45	39	27,81%	28,26%	-6	-13,33%
Classe	112	96	70,14%	69,57%	-16	-14,29%
ID	4	3	2,04%	2,17%	-1	-25,00%

Fonte: Autor

Nas Ilustrações 18, 19 e 20 são apresentados os histogramas referentes à análise da quantidade de cada um dos tipos de seletores em cada um dos *commits* de arquivos CSS nos projetos analisados. Ao todo, foram 9831 commits de arquivos CSS em 22 projetos. A partir desses histogramas é possível averiguar a concentração dos seletores ao longo do tempo dos projetos.

Na Ilustração 18 – Histograma – Seletores de Tipo, é apresentado o resultado da contagem dos Seletores de Tipo em cada um dos *commits* de arquivos CSS nos projetos analisados. Pode-se notar que na maioria dos *commits*, os arquivos possuíam uma quantidade de Seletores de Tipo entre 1 e 50. Esta faixa teve uma frequência de 5466

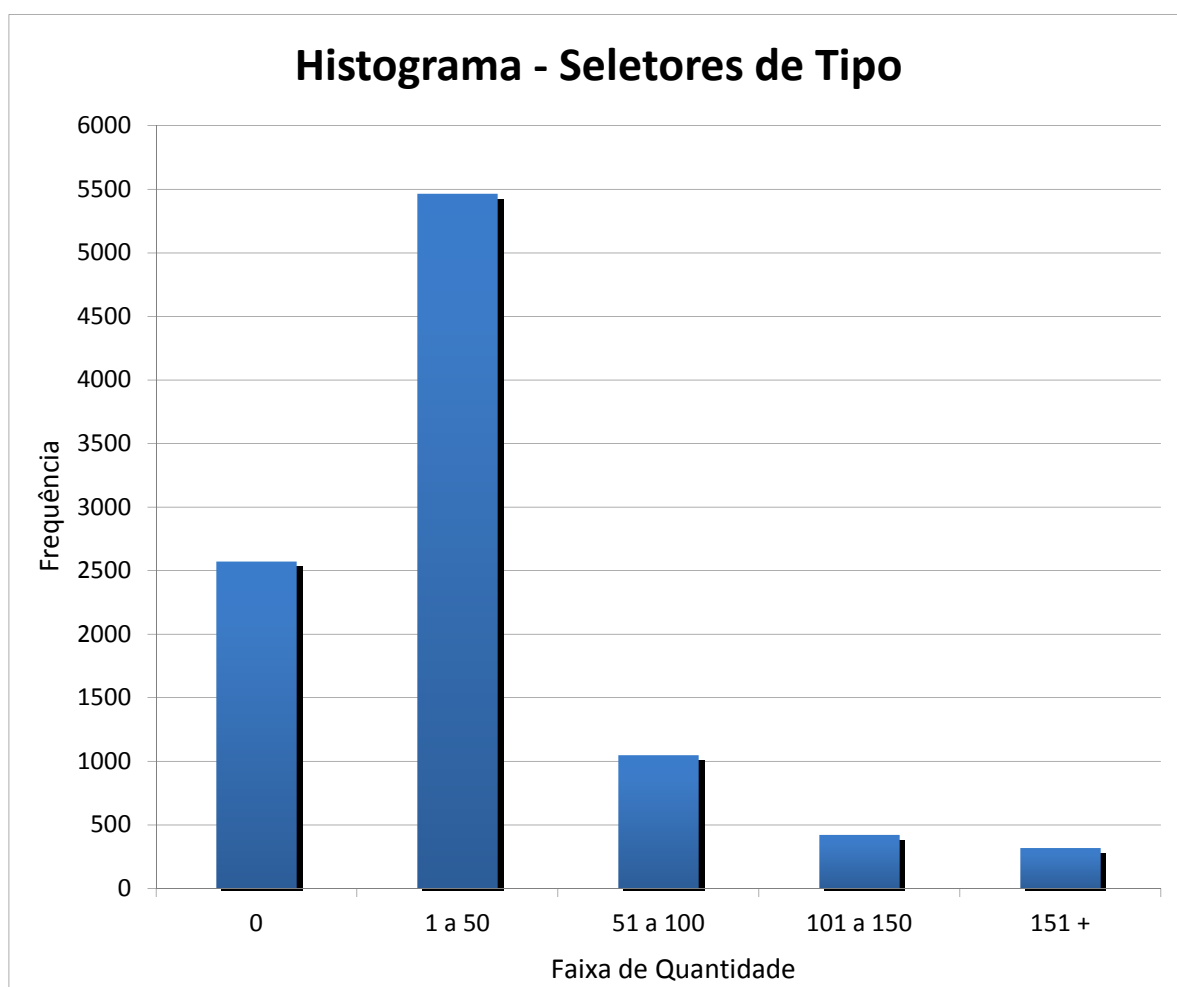


Ilustração 18 – Histograma – Seletores de Tipo

Fonte: Autor

Na Ilustração 19 – Histograma – Seletores de Classe, é apresentado o resultado da contagem dos Seletores de Classe em cada um dos *commits* de arquivos CSS nos projetos analisados. Este seletor foi o que apresentou um maior equilíbrio entre as faixas de quantidade analisadas, com exceção à faixa de 1 a 50, que teve uma frequência de 4566.

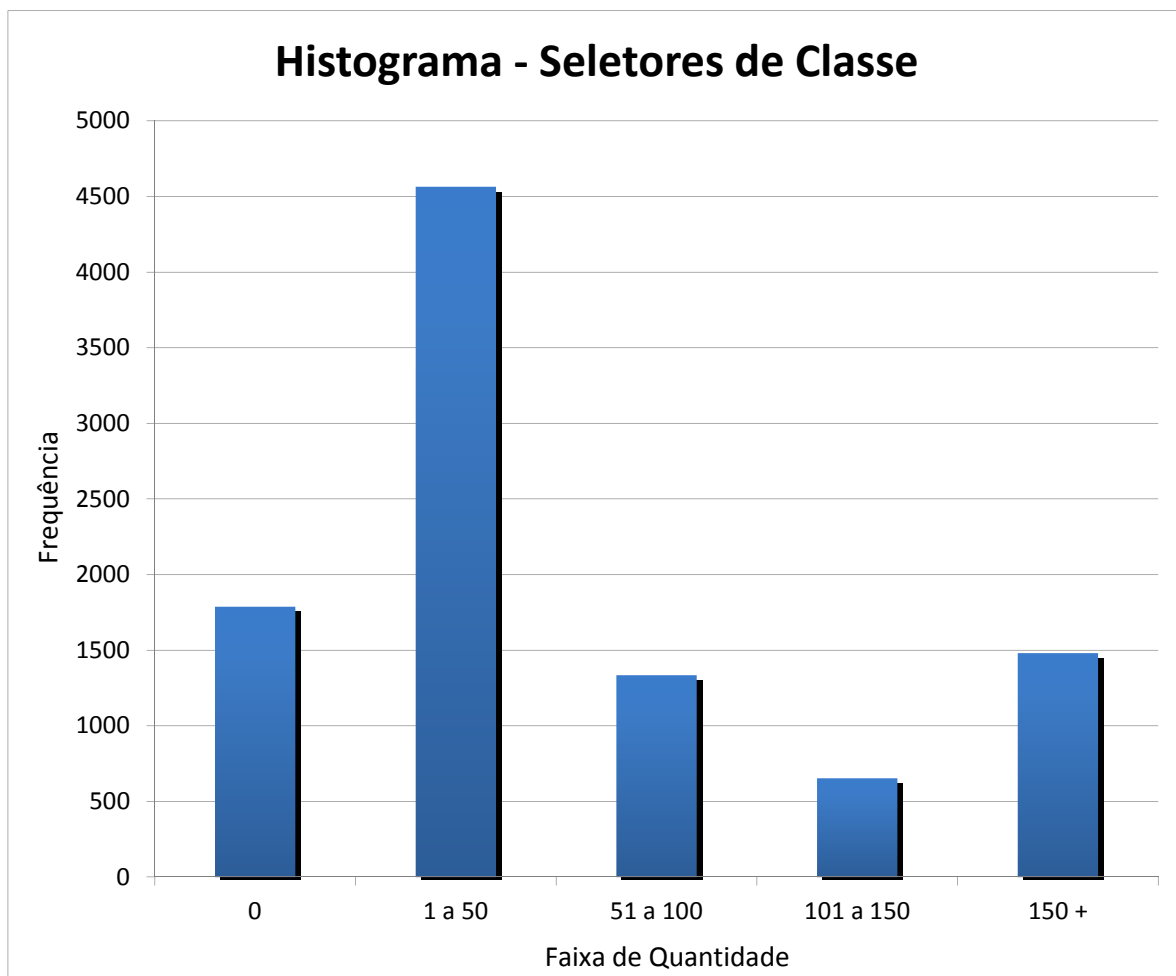


Ilustração 19 – Histograma – Seletores de Classe

Fonte: Autor

Na Ilustração 20 – Histograma – Seletores de ID, é apresentado o resultado da contagem dos Seletores de ID em cada um dos *commits* de arquivos CSS nos projetos analisados. Este seletor apresentou quase toda a sua concentração na faixa de quantidade 0 e na faixa entre 1 a 50, com apenas uma pequena frequência nas demais faixas.

A faixa de arquivos com a quantidade de Seletores de ID entre 101 a 150 apresentou uma frequência de 22 e a faixa para arquivos com 151 ou mais Seletores de ID apresentou uma frequência de apenas 10, o que demonstrou ser exceções dentre os arquivos e projetos analisados.

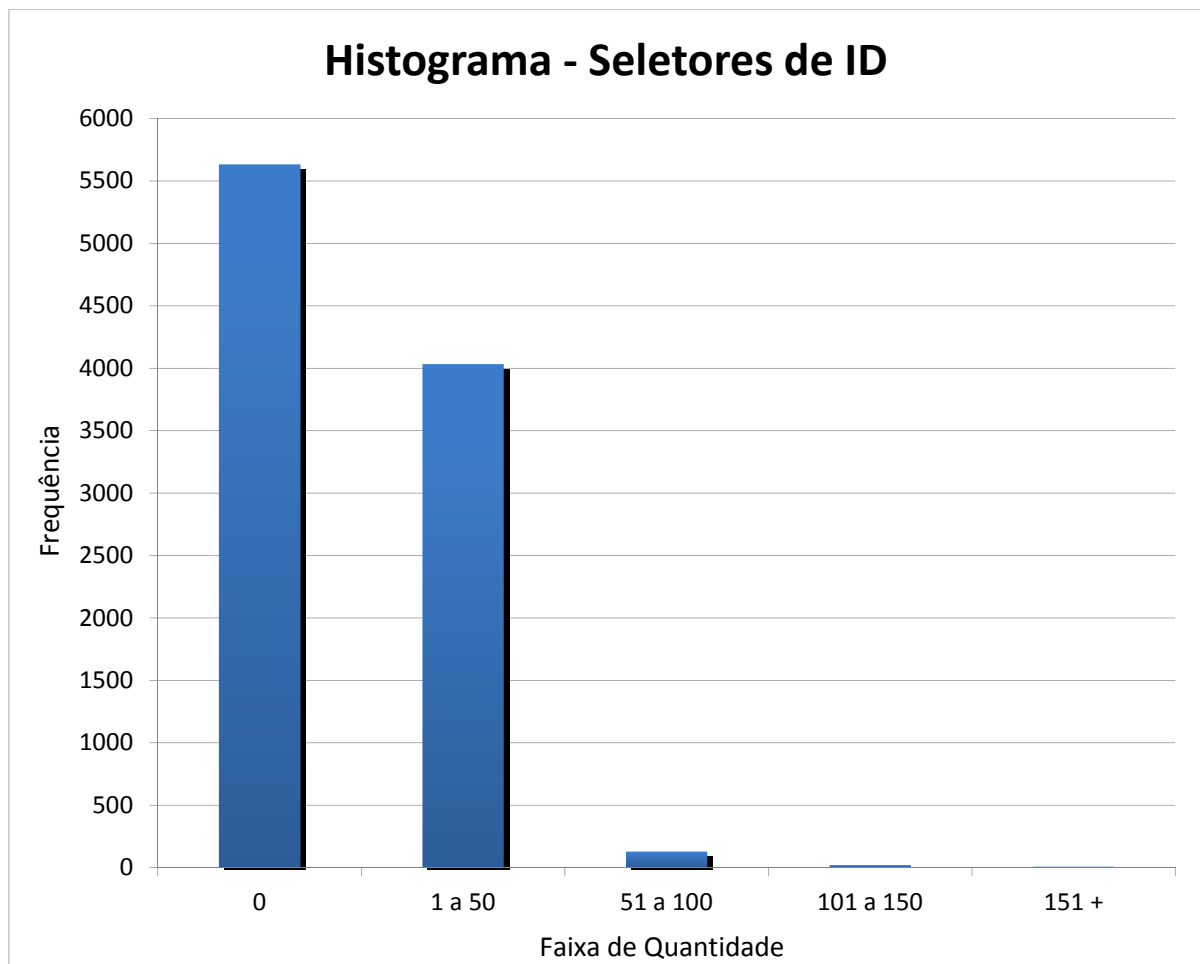


Ilustração 20 – Histograma – Seletores de ID

Fonte: Autor

8 CONSIDERAÇÕES FINAIS

Arquivos CSS são folhas de estilo que padronizam e centralizam a utilização de estilos visuais na camada de *View* em uma arquitetura MVC. Estes arquivos são compostos, basicamente, por um conjunto de regras que determinam como os estilos são aplicados. Estas regras podem utilizar-se de Seletores de Tipo, Seletores de Classe e Seletores de ID. Entretanto, para a construção de bons arquivos CSS, que possuam alto nível de manutenibilidade e reusabilidade, é necessário que a equipe de desenvolvimento se atente a alguns aspectos, como por exemplo, o uso excessivo de Seletores de ID que, devido as suas características, não possuem facilidade de reutilização e podem acarretar diretamente em uma necessidade de criação de diversos outros seletores e, conseqüentemente, maior taxa de manutenção nos arquivos CSS.

Neste trabalho foi proposto efetuar um estudo na evolução dos Seletores de Tipo, Seletores de Classe e Seletores de ID ao longo do tempo de projetos de software. Através da aplicação da métrica definida, vimos que, dentre todos os seletores analisados, o que apresentou uma maior taxa de incidência foi o Seletor de Classe, com um valor médio de 70,14% no início dos projetos e 69,57% no fim dos projetos, enquanto que o seletor menos utilizado foi o Seletor de ID, com um valor médio de 2,04% no início dos projetos e 2,17% no fim dos projetos.

Os histogramas de cada um dos seletores apresentaram, em geral, a faixa de quantidade de 1 a 50 como a maior frequência de aparição de cada seletor nos arquivos presentes nos *commits*. Para o Seletor de ID, a faixa de quantidade de 0 foi ainda maior que a faixa de 1 a 50, demonstrando, assim, a baixa utilização dos Seletores de ID.

Tendo em vista que o Seletor de Classe é o tipo de seletor mais facilmente reutilizável, enquanto que o Seletor de ID é o seletor com menos reusabilidade, e, vendo que foi utilizado, em sua maioria, Seletores de Classe, pode-se afirmar que a taxa de criação e utilização desses seletores nos projetos analisados seguiu uma tendência que proporcionou o aproveitamento e reusabilidade dos seletores criados.

Foi possível visualizar, também, uma pequena diminuição na quantidade de cada um dos seletores existentes.

Desta forma, é possível afirmar que a maneira como foram criados os seletores proporcionou uma considerável taxa de reutilização, fazendo com que a sua quantidade diminuísse ao longo do tempo do projeto e, também, gerou uma pequena necessidade de manutenção, de forma que, apenas 8% das manutenções existentes nos projetos analisados envolveram *commits* de arquivos CSS.

8.1 TRABALHOS FUTUROS

Como uma extensão da pesquisa proposta neste trabalho, a análise evolutiva dos arquivos CSS pode ser efetuada em comparação ao nível de maturidade e experiência profissional da equipe de desenvolvimento, assim como ao tempo no qual cada um dos membros já está envolvido com o projeto.

Esses dois fatores podem influenciar na qualidade do desenvolvimento e, consequentemente, nos resultados obtidos. A maturidade e experiência profissional influenciarão diretamente nas decisões e escolhas sobre como construir os arquivos CSS juntamente de seus seletores e, por sua vez, o tempo de colaboração em um projeto dará conhecimento ao desenvolvedor sobre os seletores que já existem ou que ainda necessitam ser criados, podendo assim evitar eventuais duplicidades em seletores.

REFERÊNCIAS

BUNGE, Mário. **Epistemologia: curso de atualização**. São Paulo: T. A. Queiroz: EDUSP, 1980.

BURBECK, Steve. **Applications Programming in Smalltalk-80(TM): How to use Model-View-Controller (MVC)**. mar-1997.

BUSCHMANN, Frank; MEUNIER, Regine; ROHNERT, Hans; SOMMERLAND, Peter; STAL, Michael. **Pattern-Oriented Software Architecture. A system of Patterns**. Inglaterra. Wiley, 1996.

CASCADING STYLE SHEETS. Disponível em: <<http://www.w3.org/Style/CSS/>>. Acesso em: 23 de nov. 2014.

ÇELIK, Tantek; BOS, Bert; ATTINASI, Marc. **Syntax of CSS rules in HTML's "style" attribute**. W3C Recommendation. mai-2002. Disponível em: <<http://www.w3.org/TR/2002/WD-css-style-attr-20020515>>. Acesso em: 05 de jan. de 2015.

ÇELIK, Tantek; ETERMAD, Erika J. **CSS Style Attributes**. W3C Recommendation. nov-2013. Disponível em: <<http://www.w3.org/TR/css-style-attr/>>. Acesso em: 05 de jan. de 2015.

ÇELIK, Tantek; ETERMAD, Erika J.; GLAZMAN, Daniel; HICKSON, Ian; LINSS, Peter; WILLIAMS John. **Selectors Level 3**. W3C Recommendation. set-2011. Disponível em: <<http://www.w3.org/TR/css3-selectors/>>. Acesso em: 02 de fev. de 2015.

CHAKRABARTI, Soumen; ESTER, Martin; FAYYARD, Usama; GEHRKE, Johannes; HAN, Jiawei; MORISHITA, Shinichi; PIATETSKY-SHAPIRO, Gregory; WANG, Wei. **Data Mining Curriculum: A Proposal**. v. 1. Abr, 2006.

COSTA, Carlos J., **Desenvolvimento para web**. Lisboa, Lusocredito, 2007.

ECKSTEIN, Robert. **Java SE Application Design with MVC**. 2007. Disponível em: <<http://www.oracle.com/technetwork/articles/javase/index-142890.html>>. Acesso em: 19 de jan de 2015.

FAYYARD, Usama; PIATETSKY-SHAPIRO, Gregory; SMYTH Padhraic. **From Data Mining to Knowledge Discovery in Databases**. American Association for Artificial Intelligence. AI Magazine. 1996. p. 37-54.

FRANCIS, Mark Norman. **A história da internet e da web, e a evolução dos padrões web**. out-2008.

HERZIG, Kim Sebastian; ZELLER, Andreas. **Mining your own evidence, em Making Software**. Califórnia, USA. O'Reilly, 2011. p. 517–529.

LIE, Håkon Wium; BOS, Bert. **Cascading Style Sheets, designing for the Web**. Addison Wesley, 1999.

LIE, Håkon Wium; ATKINS Jr, Tab. **CSS Cascading and Inheritance Level 3**. W3C Candidate Recommendation. out-2013. Disponível em: <<http://www.w3.org/TR/css-cascade-3/#at-import>>. Acesso em: 19 de jan. de 2015.

LONGMAN, Addison Wesley. **A history of HTML**. Disponível em: <<http://www.w3.org/People/Raggett/book4/ch02.html>>. Acesso em: 16 de nov. de 2014.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de Metodologia Científica**. 5ª Edição. São Paulo. Editora Atlas. 2003

MEYER, Eric A. **Cascading Style Sheets: The Definitive Guide**, Second Edition. 2004.

NAGAPPAN, Nachiappan; BALL, Thomas; ZELLER, Andreas. **Mining metrics to predict component failures**. In: Proceedings of the 28th international conference on Software engineering. ACM, 2006. p. 452-461.

OLIVEIRA Jr, Edson Alves de; FORTES, Renata Pontim de Mattos. **Arquitetura de Software na Web Atual: Processamento no Servidor**. São Paulo, Instituto de Ciências Matemáticas e de Computação (ICMC), Universidade de São Paulo (USP), 2010.

PHP PHP: Hypertext Preprocessor. Disponível em: <<http://php.net/>>. Acesso em: 23 de nov. de 2014.

PIETERS, Simon; THOMPSON, Henry S. **Associating Style Sheets with XML documents**. W3C Recommendation. out-2010. Disponível em: <<http://www.w3.org/TR/xml-stylesheet/>>. Acesso em: 05 de jan. 2015.

REENSKAUG, Trygve. **The original MVC reports**. Oslo, Dept. of Informatics, University of Oslo. dez-2007.

REENSKAUG, Trygve. **Models-Views-Controllers**. dez-1979.

REENSKAUG, Trygve. **The Model-View-Controller (MVC). Its Past and Present**. Oslo. University of Oslo. ago-2003.

SOKOL, Francisco. **MetricMiner: uma ferramenta web de apoio à mineração de repositórios de software**. 2012.

SURYANARAYANA, Girish; SAMARTHYAM, Ganesh; SHARMA, Tushar. **Refactoring for Software Design Smells: Managing Technical Debt**. Elsevier. 2014.